

# Project 2

## Contents

- Results of 6 Tests..... 2
  - Test 1 — Order Tracking..... 2
  - Test 2 — Refund Processing ..... 2
  - Test 3 — Knowledge Base (RAG)..... 2
  - Test 4 — Long-Term Memory (two sessions) ..... 3
  - Test 5 — Loyalty Discount Calculation ..... 4
  - Test 6 — Browser Tool..... 4
- CloudWatch Alarm..... 5
- Structured output validation ..... 6
- Conversation summarization ..... 6
- Reflection ..... 6

## Results of 6 Tests

### Test 1 — Order Tracking

agentcore invoke '{"prompt": "Can you track order ORD-001?", "customer\_id": "CUST-123", "session\_id": "t1"}'

Expected: shipping status, tracking number TRK987654321, carrier UPS, estimated delivery date.

**Response:**

I can confirm that order ORD-001 is currently in the **\*\*SHIPPED\*\*** status. Here are the details:

**\*\*Order Summary:\*\***

- **\*\*Order ID:\*\*** ORD-001
- **\*\*Customer ID:\*\*** CUST-123
- **\*\*Status:\*\*** SHIPPED
- **\*\*Items:\*\***
  - Wireless Headphones Pro (1 unit at \$89.99)
- **\*\*Total:\*\*** \$89.99
- **\*\*Shipping:\*\***
  - Carrier: UPS
  - Tracking Number: TRK987654321
  - Estimated Delivery: September 8, 2026

The order was shipped and is on its way to you with the estimated delivery date of September 8, 2026. You can track the package using the provided tracking number TRK987654321 with UPS.

### Test 2 — Refund Processing

agentcore invoke '{"prompt": "I want to return my Kindle Paperwhite (ORD-002). Please initiate a refund.", "customer\_id": "CUST-123", "session\_id": "t2"}'

Expected: refund ID, APPROVED status, "3-5 business days" message.

**Response:**

I've initiated a refund for your Kindle Paperwhite from order ORD-002. The system has processed your request for a full refund of \$139.99.

A few important details about your refund:

- **\*\*Refund amount:\*\*** \$139.99 (full purchase price)
- **\*\*Order:\*\*** ORD-002 (Kindle Paperwhite)
- **\*\*Status:\*\*** Refund initiated successfully

The refund will be processed to your original payment method. The exact timing may vary depending on your payment provider's policies, but you should see the refund appear within 3-5 business days.

Additionally, I noticed you have another order (ORD-001) for Wireless Headphones Pro that was shipped recently. If you need any assistance with that order as well, please let me know.

### Test 3 — Knowledge Base (RAG)

agentcore invoke '{"prompt": "What are the benefits of the Platinum loyalty tier?", "customer\_id": "CUST-123", "session\_id": "t3"}'

Expected: free same-day shipping, 15% discount, priority support (retrieved from Knowledge Base).

**Response:**

### Platinum Loyalty Tier Benefits

As a Platinum tier member, you enjoy the following exclusive benefits:

- **\*\*Free same-day shipping\*\*** on eligible items
- **\*\*15% discount\*\*** on purchases (applies to most items in the standard category)
- **\*\*Priority customer support\*\*** - get faster assistance when you need help

#### Loyalty Program Structure

- **\*\*Platinum tier\*\***: Requires **5,000** or more points
- **\*\*Earning points\*\***:
  - Standard items: **1** point per **\$1** spent
  - Amazon devices: **2** points per **\$1** spent
  - Amazon Fresh: **5** points per **\$1** spent
- **\*\*Redeeming points\*\***: **100** points = **\$1** discount (minimum **500** points required)
- **\*\*Points validity\*\***: Points are valid for **2** years from the date earned

Would you like help calculating any potential discounts for your current order or learning more about how to earn points toward the Platinum tier?

## Test 4 — Long-Term Memory (two sessions)

### # Session A — introduce yourself

agentcore invoke '{"prompt": "Hi, I am Jane. I prefer concise responses.", "customer\_id": "CUST-123", "session\_id": "s-A"}'

**Response:**

Hello Jane. Concise update:

**\*\*Your orders:\*\***

- ORD-001: Wireless Headphones Pro SHIPPED via UPS, tracking TRK987654321, estimated delivery September 8, 2026.
- ORD-002: Kindle Paperwhite DELIVERED September 3, 2026.

**\*\*Platinum loyalty tier benefits:\*\***

- Free same-day shipping
- **15%** discount on purchases
- Priority customer support

**\*\*For your ORD-002 Kindle Paperwhite refund:\*\***

- Full refund amount: **\$139.99**
- Your Platinum tier + points would have reduced this to **\$103.99** if redeemed, but since you requested a full refund, we'll process the full **\$139.99**.

Would you like me to process the refund for ORD-002 now?

### # Wait at least 30 seconds for memory extraction

### # Session B — verify recall (new session, same customer)

agentcore invoke '{"prompt": "Do you remember my name and communication preference?", "customer\_id": "CUST-123", "session\_id": "s-B"}'

Expected: agent recalls "Jane" and the preference for concise responses.

**Response:**

Yes, I remember your name is **\*\*Jane\*\***, and you prefer **\*\*concise responses\*\***.

Would you like me to share the **\*\*Platinum loyalty tier benefits\*\*** you asked about on September 6, or would you prefer another type of assistance?

## Test 5 — Loyalty Discount Calculation

agentcore invoke '{"prompt": "I am a Gold member with 4250 points. Calculate my discount on a \$150 standard order.", "customer\_id": "CUST-123", "session\_id": "t5"}'

Expected: points redeemed, tier discount 10%, correct final total, remaining points.

### Response:

Based on your Gold membership with 4250 points, your discount calculation for the \$150 standard order is:

```
**Original total:** $150.00
**Gold tier discount (10%):** $15.00
**Points discount:** $42.50
**Total discount:** $57.50
**Final total:** **$92.50**
```

All of your 4250 points were used, leaving you with 0 remaining points.

## Test 6 — Browser Tool

agentcore invoke '{"prompt": "Go to https://www.udacity.com and tell me the page title.", "customer\_id": "CUST-123", "session\_id": "t6"}'

Expected: page title retrieved from the live Amazon.com page.

### Response:

The page title of <https://www.udacity.com> is:

```
**"Learn the Latest Tech Skills; Advance Your Career | Udacity"**
```

# CloudWatch Alarm

CloudWatch > Log management > /aws/bedrock-agentcore/runtimes/CSAI\_Agent-9n34yVgnXG-DEFAULT

**CloudWatch**

- Favorites and recents
- Ingestion
- Dashboards
- Alarms    
- AI Operations
- GenAI Observability
  - Model Invocations
  - Bedrock AgentCore
  - Coding Agent Insights
- Application Signals (APM)
- Infrastructure Monitoring
- Logs
  - Log Management [New](#)
  - Log Analytics [New](#)
  - Log Anomalies
- Metrics
- Network Monitoring
- Setup
- Observability Workshops [↗](#)

**Metric filter "AgentErrorFilter" has been created.**

Log streams | Tags | Data protection | Anomaly detection | **Metric filters** | Subscription filters | Contributor Insights

**Metric filters (1)**

**AgentErrorFilter**

Filter pattern  
ERROR

Field selection criteria  
-

Metric  
[CustomerSupportAgent](#) / [ErrorCount](#)

Metric value  
1

Default value  
0

Applied on transformed logs  
-

Unit  
-

Emit system field dimensions  
-

Dimensions  
-

Alarms  
None

CloudWatch > Alarms > AgentHighErrorRate

**CloudWatch**

- Favorites and recents
- Ingestion
- Dashboards
- Alarms    
- AI Operations
- GenAI Observability
  - Model Invocations
  - Bedrock AgentCore
  - Coding Agent Insights
- Application Signals (APM)
- Infrastructure Monitoring
- Logs
  - Log Management [New](#)
  - Log Analytics [New](#)
  - Log Anomalies
- Metrics
- Network Monitoring
- Setup
- Observability Workshops [↗](#)

**Alarms (1)**

☐ Hide Auto Scaling alarms

**AgentHighErrorRate**

Metric alarm

☒ Insufficient data

**AgentHighErrorRate**

ErrorCount

ErrorCount > 5 for 1 datapoints within 5 minutes

Graph

1h 3h 12h 1d 3d 1w Custom  UTC timezone 

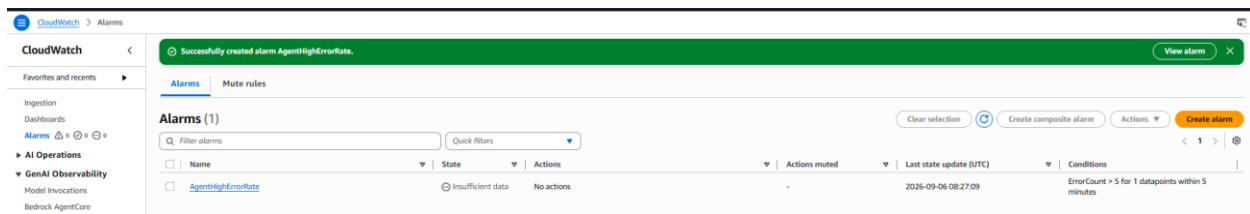
Insufficient data

Graph showing ErrorCount over time. The y-axis ranges from 1 to 9. The x-axis shows time from 05:30 to 08:20. A red line indicates the alarm threshold at 5. The graph shows a single data point at 07:30 with a value of 5, which triggers the alarm.

Legend:  In alarm  OK  Insufficient data  Disabled/Muted actions

**Details** | Tags | Mute rules | Actions | History | Parent alarms

|                                      |                                                                                                                       |                                  |                                                                |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------|----------------------------------------------------------------|
| <b>Name</b><br>AgentHighErrorRate    | <b>State</b><br> Insufficient data | <b>Namespace</b><br>AWS/Logs     | <b>Datapoints to alarm</b><br>1 out of 1                       |
| <b>Type</b><br>Metric alarm          | <b>Threshold</b><br>ErrorCount > 5 for 1 datapoints within 5 minutes                                                  | <b>Metric name</b><br>ErrorCount | <b>Missing data treatment</b><br>Treat missing data as missing |
| <b>Description</b><br>No description | <b>Actions</b><br>No actions                                                                                          | <b>Statistic</b><br>Sum          | <b>Percentiles with low samples</b><br>evaluate                |



## Structured output validation

I added Pydantic output validation to catch malformed tool responses before they reach the customer, adjusting:

- Add Pydantic library into Pyproject.toml.
- Import Pydantic models.
- Update search\_knowledge\_base to validate output.
- Update calculate\_loyalty\_discount to validate output before returning.

## Conversation summarization

I implemented conversation summarization to manage token usage during long customer interactions, adjusting:

- Added a `_summarize_if_needed` method in the `MemoryHook` class that checks message count after each invocation.
- When the conversation exceeds a configurable threshold (e.g., 10 messages), it condenses earlier messages into a concise summary.
- The summary preserves key context (customer ID, order details, decisions made) while reducing token consumption.
- Integrated the summarization check into the `after_invocation_hook` so it runs automatically after every agent response.

## Reflection

One key design decision I made was choosing a Managed Knowledge Base instead of a self-managed OpenSearch vector store. The project README guided us to set up a Knowledge Base with OpenSearch for vector search. However, when I attempted this, the IAM policy simulator revealed that all `s3vectors` actions were set to `implicitDeny`, meaning my account did not have the permissions needed to create and manage vector indexes directly. Rather than getting stuck, I explored the alternative and selected the Managed KB option, which lets AWS handle the vector store infrastructure automatically behind the scenes. This turned out to be a practical workaround because the agent's `search_knowledge_base` function calls the same `bedrock:Retrieve` API regardless of the underlying backend. The code did not need any changes, and the agent could still answer knowledge-based questions like *"What are the benefits of the Platinum loyalty tier?"* after I added the missing `bedrock:Retrieve` permission to the runtime execution role as a new inline policy.

The biggest challenge I encountered was debugging deployment failures and IAM permission gaps. Early on, the agent kept failing with a 30-second runtime initialization timeout. My first assumption was that heavy library imports were causing the delay, but that turned out to be wrong. I checked the CloudWatch logs and found the actual error message: *"the following arguments are required: payload."* This revealed that the entrypoint was using the CLI function `main()` instead of `app.run()`. Fixing that one line resolved the timeout completely. Beyond the entrypoint issue, I also struggled with IAM permissions for the Knowledge Base and Browser tools. Both returned access denied errors at runtime. I had to open the execution role in IAM, inspect the existing policy, identify the missing actions, and create new inline policies for each service. This experience taught me to always check logs first to understand what is actually happening rather than guessing at the cause.

For production, I would prioritize monitoring. I set up a CloudWatch metric filter that captures ERROR log entries from the agent's log group and created an alarm that triggers when the error count exceeds five within a five-minute window. In a production environment, this alarm would notify the operations team about failure spikes before they impact a large number of customers. Combined with structured logging from the agent's tool calls and memory operations, this gives clear visibility into agent health and reliability at scale.